A second life starts with a first look.

Six-class waste image classification using a convolutional neural network

Measured outcome	Held-out test
Accuracy	84.43% (320 / 379 correct)
Macro F1	82.61%
Data	2,524 images; six material classes
Model	MobileNetV3-Small CNN with transfer learning
Evaluation	Stratified train/validation/test split

Prepared 11 November 2026. Course context: Deep Learning, International Program in Informatics, Universitas Islam Indonesia, odd semester 2026/2027.

This report follows the five sections in the exam brief. All reported experimental values were produced by the included implementation.

Author: Abdullah Al-Hamidi | Student ID: 24523229

GitHub repository: https://github.com/alhamidi1/sortloop

1. Project & case study

Problem and objective

This project studies supervised image classification for six waste categories: cardboard, glass, metal, paper, plastic and trash. Its objective is to train and evaluate a convolutional neural network that predicts the dataset label for one image.

Success means a reproducible training procedure, performance above a simple majority-class baseline on a held-out test split, and a careful analysis of class-level strengths and errors.

Why CNN instead of YOLO

TrashNet provides one image-level label per photograph, not bounding boxes. A CNN classifier directly matches those labels. YOLO would be appropriate for locating multiple objects in a scene, but would require representative multi-object images and box annotations. CNN classification is therefore the better match to this dataset and project objective.

Scope

The experiment predicts one of six known labels for each image. It does not determine local recyclability rules, identify hazardous materials, or establish performance in a real collection facility.

2. Data & preprocessing

Source: TrashNet by Gary Thung and Mindy Yang (2016), downloaded from the public GitHub repository. Original resized archive: 2,527 images, largely photographed on white posterboard under sunlight or room lighting. Original class counts: cardboard 403, glass 501, metal 410, paper 594, plastic 482, trash 137. The upstream dataset license and attribution are retained in the project files.

SHA-256 hashes identify and remove three exact duplicate files before splitting. The resulting 2,524 images are stratified into 1,766 training, 379 validation and 379 test examples, using seed 42. The explicit file lists are saved in ml/splits.json. Augmentations are applied after splitting and only to training examples.

Material	Train	Validation	Test
cardboard	282	61	60
glass	350	75	76
metal	286	62	61
paper	416	89	89
plastic	336	72	72
trash	96	20	21

Input pipeline

Decode to RGB; resize to 160 x 160; convert pixel values from 0-255 to 0-1; normalize per channel using means (0.485, 0.456, 0.406) and standard deviations (0.229, 0.224, 0.225). These match ImageNet pretraining statistics. Direct resizing changes aspect ratio but provides a consistent model input shape.

For each training image, cache features from one unaugmented view and two independently randomized views. Augmentation first resizes to 180 x 180, samples a RandomResizedCrop to 160 with scale 0.75-1.0, applies a random horizontal flip, then ColorJitter with brightness/contrast 0.15, saturation 0.10 and hue 0.02. This tests moderate pose and lighting variation without changing material labels. Cached augmentations are fixed for all 80 head-training epochs.

Limitations: exact deduplication does not remove near duplicates or identify multiple photos of the same physical object. The random image split may be optimistic. A future evaluation should group images by object and capture session.

3. Model implementation

Transfer learning uses Torchvision MobileNetV3-Small with IMAGENET1K_V1 weights. The original ImageNet classifier is discarded. The frozen feature stack, adaptive global average pooling and flatten produce 576 features. A new linear layer maps 576 features to six logits. Softmax is applied only for display/inference; cross-entropy consumes raw logits.

Component	Implementation	Purpose
Stem	3x3 Conv, stride 2, 3 to 16 channels; BatchNorm; Hardswish	Early spatial feature extraction
11 inverted residual blocks	1x1 expansion; 3x3 or 5x5 depthwise convolution; squeeze-excitation where configured; 1x1 projection	Efficient hierarchical visual features
Final feature convolution	1x1 Conv, 96 to 576; BatchNorm; Hardswish	Build compact feature representation
Pooling + flatten	Adaptive average pooling to 1x1; flatten	576-dimensional vector
Trained head	Linear(576, 6), with bias	Learn the six material categories

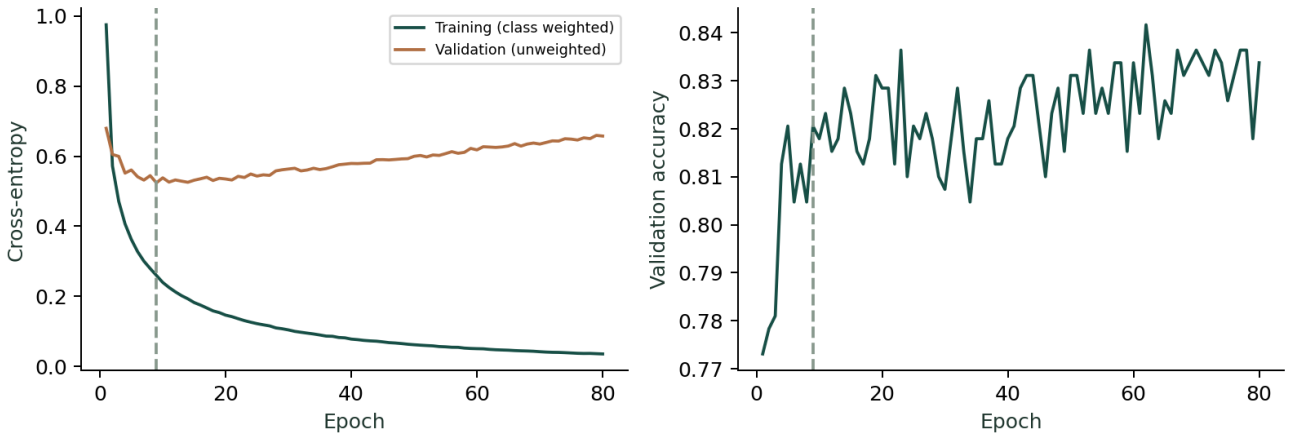
The exact model contains 52 Conv2d modules (including squeeze-excitation convolutions), one adaptive global pooling stage in the main path and one final Linear classifier. BatchNorm, ReLU, Hardswish and Hardsigmoid activations follow the Torchvision implementation. Squeeze-excitation blocks additionally use their own internal pooling. The complete instantiated architecture is included as ml/architecture.txt.

Total parameters: 930,470. Frozen CNN parameters: 927,008. Trained head parameters: 3,462 = 576 x 6 + 6. Freezing the extractor reduces compute and overfitting risk on this small dataset, at the cost of limited domain adaptation.

Implementation is in ml/train.py, reproduced in Appendix A. The exported ONNX model provides a portable copy of the trained network; the PyTorch model and export-parity record are included with the source.

4. Training & evaluation

Setting	Value
Optimizer	AdamW; learning rate 0.003; weight decay 0.02
Loss	Class-weighted cross-entropy for training; unweighted validation cross-entropy
Class weighting	N_train / (6 x count_class)
Batch size / epochs	128 feature vectors / 80 epochs
Checkpoint selection	Minimum validation loss; selected epoch 9
Randomness	Python, NumPy and Torch seeds = 42
Compute	CPU, 3 Torch threads; measured training/extraction time 50 seconds
Evaluation	One final held-out test evaluation after checkpoint selection



The dashed line marks the selected checkpoint. Later validation accuracy occasionally rises, but validation loss is the preselected selection criterion. Weighted training loss and unweighted validation loss differ in weighting and data augmentation, so their absolute levels should not be compared as identical objectives.

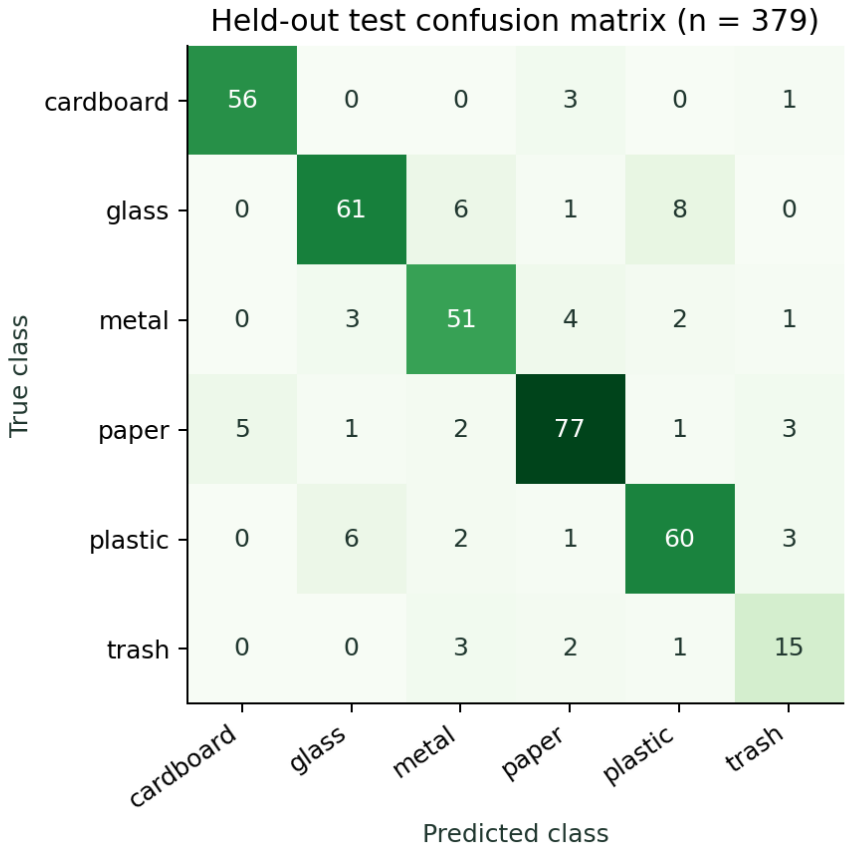
Metrics

Accuracy = correct / total. For each class, precision = TP / (TP + FP), recall = TP / (TP + FN), and F1 = 2PR / (P + R). Macro F1 averages the six class F1 scores equally, preventing the large paper class from dominating the summary.

Class	Precision	Recall	F1	Test n
cardboard	0.918	0.933	0.926	60
glass	0.859	0.803	0.830	76

Class	Precision	Recall	F1	Test n
metal	0.797	0.836	0.816	61
paper	0.875	0.865	0.870	89
plastic	0.833	0.833	0.833	72
trash	0.652	0.714	0.682	21
Macro average	0.822	0.831	0.826	379

4. Evaluation, continued



The classifier correctly predicts 320 of 379 test images (84.43%). A majority-class baseline that always predicts paper would achieve 89/379 = 23.48% accuracy on this test set. The CNN clearly outperforms that simple baseline, although stronger trained baselines were not tested.

A Wilson 95% binomial confidence interval for test accuracy is approximately 80.4%-87.7%. This interval describes finite-sample uncertainty conditional on the split; it does not account for domain shift, repeated objects or model-selection uncertainty.

Cardboard is strongest (F1 0.926). Trash is weakest (F1 0.682), with only 21 test examples and a broad, heterogeneous class. Eight glass examples are mistaken for plastic and six for metal. Transparent surfaces, reflections and similar shapes are plausible explanations, but systematic image-level error analysis is required to establish causes.

The ONNX export matches PyTorch with maximum logit error below 0.0001. This numerical parity check compares the exported model with PyTorch on identical input tensors.

5. Analysis & conclusion

What the results support

The project meets its technical objective on this dataset split: a supervised CNN was trained on relevant image labels and evaluated on an unseen test set. Its 84.43% accuracy and 82.61% macro F1 are substantially above the 23.48% majority-class baseline.

Limitations

The dataset is small and consists largely of isolated items on plain backgrounds. The random image split may be optimistic because near-duplicate or repeated-object images may remain across subsets. These results do not establish performance in a real sorting facility. The classifier is closed-set and always predicts one of the six known labels.

The heterogeneous trash category is the weakest class (F1 0.682) and has only 21 test examples. Its broad label and low sample count limit the conclusions we can draw. Class-level precision, recall, F1 and confusion-matrix errors provide a more complete picture than accuracy alone.

Future improvements

Priority	Next experiment	Evaluation focus
1	Collect more diverse, consistently labeled images	Per-class F1 and support
2	Group splits by object and capture session	Reduce near-duplicate leakage
3	Clarify or refine the broad trash label	Label consistency and false positives
4	Fine-tune upper CNN layers and compare with frozen baseline	Validation-selected model; untouched test set
5	Evaluate on independently collected images	Performance under new backgrounds and lighting

Conclusion: the CNN meets the stated image-classification objective on the held-out TrashNet split and clearly outperforms the majority baseline. Further data and evaluation are needed before making claims about real-world sorting performance.

Reproducibility and implementation checks

The full training and evaluation code is in ml/train.py and Appendix A. Reproduce the experiment by installing ml/requirements.txt, downloading the TrashNet resized dataset, and running DATASET_DIR=/path/to/dataset-resized python ml/train.py from the project root. The script records the deterministic split, epoch history, metrics, model weights and ONNX export.

Recorded checks

The documented run completed on CPU, selected the checkpoint by validation loss, then evaluated once on the held-out test set. The ONNX export was compared with PyTorch logits and passed the recorded parity check. The report charts and metric values are generated from the saved evaluation artifacts.

Limits of reproducibility

The exact file lists and seed are recorded, but the random image split may still contain near-duplicate or repeated-object images across subsets. Hardware and dependency versions can affect training time and may cause small numerical differences. Test results describe this dataset and split only; they are not a field-performance estimate.

Project source

The complete project source, model artifacts, dataset attribution, and report-generation code are available at https://github.com/alhamidi1/sortloop. The raw dataset is downloaded separately and is not bundled.

Sources & attribution

TrashNet

Gary Thung and Mindy Yang, 2016. Dataset, collection description and upstream license.

https://github.com/garythung/trashnet

MobileNetV3 paper

Howard et al., 2019. Searching for MobileNetV3.

https://arxiv.org/abs/1905.02244

Torchvision MobileNetV3

Architecture and pretrained model implementation.

https://docs.pytorch.org/vision/stable/models/mobilenetv3.html

Assignment document

Soal UTS Ganjil 2026 2027 - Deep Learning; five-section assessment structure.

User-provided PDF

Source access date: 11 November 2026. The report distinguishes this project's measured results from the original TrashNet authors' experiments.

Appendix A / training code

Complete authoritative training, evaluation and export script. Lines wrap for print; use ml/train.py from the source kit to run the code.

```
"""Train a reproducible frozen-MobileNetV3 CNN + learned linear classifier."""
import os, json, random, hashlib, time, shutil
from pathlib import Path
import numpy as np
import torch
from torch import nn
from torchvision import models, transforms
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

ROOT=Path(__file__).resolve().parents[1];
DATA=Path(os.environ.get('DATASET_DIR','/workspace/work/dataset-resized'))
OUT=ROOT/'dist'; (OUT/'model').mkdir(exist_ok=True); (OUT/'samples').mkdir(exist_ok=True)
random.seed(42); np.random.seed(42); torch.manual_seed(42); torch.set_num_threads(3)
classes=['cardboard','glass','metal','paper','plastic','trash']
files=[]; labels=[]; seen=set(); duplicates=[]
for i,c in enumerate(classes):
 for p in sorted((DATA/c).glob('*.jpg')):
  digest=hashlib.sha256(p.read_bytes()).hexdigest()
  if digest in seen: duplicates.append(str(p)); continue
  seen.add(digest); files.append(p); labels.append(i)
idx=np.arange(len(files));
train_idx,temp=train_test_split(idx,test_size=.30,stratify=labels,random_state=42)
val_idx,test_idx=train_test_split(temp,test_size=.5,stratify=np.array(labels)[temp],random_state=42)
json.dump({k:[str(files[i].relative_to(DATA)) for i in ids] for k,ids in [('train',train_idx),('vali
dation',val_idx),('test',test_idx)]},open(ROOT/'ml/splits.json','w'),indent=2)
base=models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.IMAGENET1K_V1).eval()
encoder=nn.Sequential(base.features,base.avgpool,nn.Flatten()).eval()
for p in encoder.parameters(): p.requires_grad=False
norm=transforms.Normalize([.485,.456,.406],[.229,.224,.225])
plain=transforms.Compose([transforms.Resize((160,160)),transforms.ToTensor(),norm])
aug=transforms.Compose([transforms.Resize((180,180)),transforms.RandomResizedCrop(160,scale=(.75,1.)
),transforms.RandomHorizontalFlip(),transforms.ColorJitter(.15,.15,.10,.02),transforms.ToTensor(),no
rm])
def extract(ids,tf):
 chunks=[]
 for start in range(0,len(ids),32):
  batch=torch.stack([tf(Image.open(files[i]).convert('RGB')) for i in ids[start:start+32]])
  with torch.inference_mode(): chunks.append(encoder(batch))
 return torch.cat(chunks)
start=time.time(); print('Extracting frozen CNN features',flush=True)
x=torch.cat([extract(train_idx,plain),extract(train_idx,aug),extract(train_idx,aug)])
y=torch.tensor(np.tile(np.array(labels)[train_idx],3)); xv=extract(val_idx,plain);
yv=torch.tensor(np.array(labels)[val_idx]); xt=extract(test_idx,plain)
head=nn.Linear(576,6); opt=torch.optim.AdamW(head.parameters(),lr=.003,weight_decay=.02)
weights=torch.tensor(len(train_idx)/(6*np.bincount(np.array(labels)[train_idx])),dtype=torch.float32
)
lossfn=nn.CrossEntropyLoss(weight=weights); history=[]; best=1e9
for epoch in range(1,81):
 head.train(); total=0
 for ids in torch.randperm(len(x)).split(128):
  opt.zero_grad(); loss=lossfn(head(x[ids]),y[ids]); loss.backward(); opt.step();
total+=loss.item()*len(ids)
 head.eval()
```

```
 with torch.no_grad():
  logits=head(xv); vl=nn.functional.cross_entropy(logits,yv).item();
va=(logits.argmax(1)==yv).float().mean().item()
 history.append({'epoch':epoch,'train_loss':total/len(x),'val_loss':vl,'val_accuracy':va})
 if vl<best: best=vl; best_epoch=epoch; torch.save(head.state_dict(),ROOT/'ml/head.pt')
 if epoch%10==0: print('Epoch',epoch,'validation accuracy',round(va,4),flush=True)
head.load_state_dict(torch.load(ROOT/'ml/head.pt',weights_only=True));
model=nn.Sequential(encoder,head).eval()
with torch.no_grad(): pred=head(xt).argmax(1).numpy()
yt=np.array(labels)[test_idx];
report=classification_report(yt,pred,target_names=classes,output_dict=True,zero_division=0)
metrics={'classes':classes,'dataset_count':len(files),'exact_duplicates_removed':len(duplicates),'sp
lit':{'train':len(train_idx),'validation':len(val_idx),'test':len(test_idx)},'seed':42,'best_epoch':
best_epoch,'accuracy':accuracy_score(yt,pred),'report':report,'confusion_matrix':confusion_matrix(yt
,pred).tolist(),'history':history,'seconds':round(time.time()-start),'parameters':sum(p.numel() for
p in model.parameters()),'trainable_parameters':sum(p.numel() for p in head.parameters())}
json.dump(metrics,open(OUT/'metrics.json','w'),indent=2)
json.dump({'classes':classes,'size':160,'mean':[.485,.456,.406],'std':[.229,.224,.225]},open(OUT/'mo
del/config.json','w'),indent=2)
torch.save(model.state_dict(),ROOT/'ml/model.pt')
torch.onnx.export(model,torch.zeros(1,3,160,160),str(OUT/'model/sortloop.onnx'),input_names=['image'
],output_names=['logits'],opset_version=17,dynamo=False)
samples=[]
for c in range(6):
 i=next(i for i in test_idx if labels[i]==c); dest=OUT/'samples'/f'{classes[c]}.jpg';
shutil.copy(files[i],dest); samples.append({'class':classes[c],'file':f'samples/{classes[c]}.jpg','s
ource':str(files[i].relative_to(DATA))})
json.dump(samples,open(OUT/'samples/index.json','w'),indent=2)
# Verify exported inference against the training framework.
import onnxruntime as ort
session=ort.InferenceSession(str(OUT/'model/sortloop.onnx'));
image=plain(Image.open(files[test_idx[0]]).convert('RGB')).unsqueeze(0)
with torch.no_grad(): expected=model(image).numpy()
actual=session.run(None,{'image':image.numpy()})[0]; delta=float(np.max(np.abs(expected-actual)));
assert delta<1e-4
json.dump({'max_logit_error':delta,'passed':True},open(ROOT/'ml/export-check.json','w'),indent=2)
print(json.dumps({k:v for k,v in metrics.items() if k not in
['history','report','confusion_matrix']}),flush=True)
```

Appendix B / convolution map

Every convolution in the deployed CNN feature extractor. BN follows feature-block convolutions; squeeze-excitation uses internal 1x1 convolutions. Depthwise layers have groups equal to input channels.

Module	In / out	Kernel	Stride	Groups
0.0.0	3 / 16	(3, 3)	(2, 2)	1
0.1.block.0.0	16 / 16	(3, 3)	(2, 2)	16
0.1.block.1.fc1	16 / 8	(1, 1)	(1, 1)	1
0.1.block.1.fc2	8 / 16	(1, 1)	(1, 1)	1
0.1.block.2.0	16 / 16	(1, 1)	(1, 1)	1
0.2.block.0.0	16 / 72	(1, 1)	(1, 1)	1
0.2.block.1.0	72 / 72	(3, 3)	(2, 2)	72
0.2.block.2.0	72 / 24	(1, 1)	(1, 1)	1
0.3.block.0.0	24 / 88	(1, 1)	(1, 1)	1
0.3.block.1.0	88 / 88	(3, 3)	(1, 1)	88
0.3.block.2.0	88 / 24	(1, 1)	(1, 1)	1
0.4.block.0.0	24 / 96	(1, 1)	(1, 1)	1
0.4.block.1.0	96 / 96	(5, 5)	(2, 2)	96
0.4.block.2.fc1	96 / 24	(1, 1)	(1, 1)	1
0.4.block.2.fc2	24 / 96	(1, 1)	(1, 1)	1
0.4.block.3.0	96 / 40	(1, 1)	(1, 1)	1
0.5.block.0.0	40 / 240	(1, 1)	(1, 1)	1
0.5.block.1.0	240 / 240	(5, 5)	(1, 1)	240
0.5.block.2.fc1	240 / 64	(1, 1)	(1, 1)	1
0.5.block.2.fc2	64 / 240	(1, 1)	(1, 1)	1

Module	In / out	Kernel	Stride	Groups
0.5.block.3.0	240 / 40	(1, 1)	(1, 1)	1
0.6.block.0.0	40 / 240	(1, 1)	(1, 1)	1
0.6.block.1.0	240 / 240	(5, 5)	(1, 1)	240
0.6.block.2.fc1	240 / 64	(1, 1)	(1, 1)	1
0.6.block.2.fc2	64 / 240	(1, 1)	(1, 1)	1
0.6.block.3.0	240 / 40	(1, 1)	(1, 1)	1
0.7.block.0.0	40 / 120	(1, 1)	(1, 1)	1
0.7.block.1.0	120 / 120	(5, 5)	(1, 1)	120
0.7.block.2.fc1	120 / 32	(1, 1)	(1, 1)	1
0.7.block.2.fc2	32 / 120	(1, 1)	(1, 1)	1
0.7.block.3.0	120 / 48	(1, 1)	(1, 1)	1
0.8.block.0.0	48 / 144	(1, 1)	(1, 1)	1
0.8.block.1.0	144 / 144	(5, 5)	(1, 1)	144
0.8.block.2.fc1	144 / 40	(1, 1)	(1, 1)	1
0.8.block.2.fc2	40 / 144	(1, 1)	(1, 1)	1
0.8.block.3.0	144 / 48	(1, 1)	(1, 1)	1
0.9.block.0.0	48 / 288	(1, 1)	(1, 1)	1
0.9.block.1.0	288 / 288	(5, 5)	(2, 2)	288
0.9.block.2.fc1	288 / 72	(1, 1)	(1, 1)	1
0.9.block.2.fc2	72 / 288	(1, 1)	(1, 1)	1
0.9.block.3.0	288 / 96	(1, 1)	(1, 1)	1
0.10.block.0.0	96 / 576	(1, 1)	(1, 1)	1
0.10.block.1.0	576 / 576	(5, 5)	(1, 1)	576
0.10.block.2.fc1	576 / 144	(1, 1)	(1, 1)	1
0.10.block.2.fc2	144 / 576	(1, 1)	(1, 1)	1

Module	In / out	Kernel	Stride	Groups
0.10.block.3.0	576 / 96	(1, 1)	(1, 1)	1
0.11.block.0.0	96 / 576	(1, 1)	(1, 1)	1
0.11.block.1.0	576 / 576	(5, 5)	(1, 1)	576
0.11.block.2.fc1	576 / 144	(1, 1)	(1, 1)	1
0.11.block.2.fc2	144 / 576	(1, 1)	(1, 1)	1
0.11.block.3.0	576 / 96	(1, 1)	(1, 1)	1
0.12.0	96 / 576	(1, 1)	(1, 1)	1